VOL. 48

Fine-tuning

Adapting pre-trained language models to specific tasks

Source	huggingface.co/docs/transformers
Generated	2026-10-11
Level	Intermediate
Estimated study time	20 hours
Language	English
Volume	Vol. 48 - Fine-tuning Transformers

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume **3**

Chapter 1 - What is fine-tuning? **3**

1.1 Summary 3

1.2 Concepts 3

1.3 Mini test 3

Chapter 2 - The fine-tuning workflow **3**

2.1 Summary 3

2.2 Examples 4

Example 1 - Text classification fine-tuning 4

2.3 Mini test 4

Chapter 3 - Evaluation and metrics **5**

3.1 Summary 5

3.2 Examples 5

3.3 Mini test 5

Final test **5**

4.1 Final test answers 5

Sources **6**

How to use this volume

Prerequisites. Familiarity with Hugging Face Transformers (Vol 47) and Python.

Estimated study time. About 20 hours.

Chapter 1 - What is fine-tuning?

1.1 Summary

- Fine-tuning trains a pre-trained model on task-specific data.
- Usually requires much less data than pre-training (hundreds to thousands of examples).
- Benefits: better performance, domain adaptation, task specialization.
- Costs: compute, time, storage for each variant.

1.2 Concepts

Term	Definition
Pre-training	Large-scale training on generic data
Fine-tuning	Small-scale training on specific task
Epochs	Full passes through training data
Batch size	Examples per gradient update
Learning rate	Step size for weight updates (2e-5 to 5e-5)

1.3 Mini test

1. How much data is needed for fine-tuning? 2. What is typical learning rate? 3. Why fine-tune?

Answers: 1) Hundreds to thousands of labeled examples. 2) 2e-5 to 5e-5. 3) To adapt the model to specific tasks/domains.

Chapter 2 - The fine-tuning workflow

2.1 Summary

1. Prepare a labeled dataset.
2. Tokenize inputs.

3. Create DataLoaders.
4. Set up training arguments.
5. Initialize Trainer.
6. Train and evaluate.
7. Save and deploy.

2.2 Examples

Example 1 - Text classification fine-tuning

Python

```python
# ch02_training.py
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from datasets import Dataset

model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Sample data
train_data = [
    {"text": "This movie is great", "label": 1},
    {"text": "Terrible quality", "label": 0},
    {"text": "Highly recommend", "label": 1},
    {"text": "Waste of money", "label": 0},
]

train_ds = Dataset.from_list(train_data)

def tokenize(example):
    return tokenizer(example["text"], padding="max_length", truncation=True)

tokenized_ds = train_ds.map(tokenize, batched=True)

model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=8,
    learning_rate=2e-5,
    weight_decay=0.01,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_ds,
    eval_dataset=tokenized_ds,  # use same for demo
)

# trainer.train()
# model.save_pretrained("./my-finetuned-model")
```

2.3 Mini test

1. What is a TrainingArgument? 2. What is the Trainer class? 3. How to save a model?

Answers: 1) Configuration for training run. 2) A high-level API for training. 3) `model.save_pretrained(path)`.

Chapter 3 - Evaluation and metrics

3.1 Summary

- Evaluate on a held-out test set.
- Use task-appropriate metrics: accuracy, F1, BLEU, ROUGE.
- The `evaluate` library provides standardized metrics.
- Log metrics for comparison across runs.

3.2 Examples

Python

```python
# ch03_eval.py
import evaluate

accuracy = evaluate.load("accuracy")
results = accuracy.compute(predictions=[0, 1, 0, 1], references=[0, 1, 1, 1])
print(f"Accuracy: {results['accuracy']:.3f}")
```

3.3 Mini test

1. What does the evaluate library do? 2. Why use a test set? 3. What metrics for classification?

Answers: 1) Provides standardized evaluation metrics. 2) To get unbiased performance estimates. 3) Accuracy, precision, recall, F1.

Final test

1. What is the typical learning rate for fine-tuning?
2. What does `num_train_epochs` control?
3. What is the difference between pre-training and fine-tuning?
4. How do you save a fine-tuned model?
5. What is the Trainer API?

4.1 Final test answers

1. 2e-5 to 5e-5.

2. How many times the model sees the full dataset.
3. Pre-training is large-scale; fine-tuning is small-scale and task-specific.
4. `model.save_pretrained(path)`.
5. A high-level training abstraction in transformers.

Sources

- https://huggingface.co/docs/transformers/training
- https://huggingface.co/docs/transformers/main_classes/trainer
- https://huggingface.co/docs/evaluate